

Hotel Reservation System Design

By Lang Wang

Step 1 - Understand the Problem and Establish Design Scope

Functional Requirements

- View hotel/room detail page
- View booking confirmation page
- Make reservations (full payment required upfront)
- Admin panel to add/update/delete hotel or room data
- Support overbooking up to 10%

Non-Functional Requirements

- High concurrency (especially during events or peak season)
- Moderate latency (a few seconds acceptable for reservations)

Constraints

- 5,000 hotels, 1 million rooms total
- Average daily reservations: ~240,000 (TPS ~3)
- Traffic funnel:
 - Hotel/Room Detail View: 300 QPS
 - Booking Page View: 30 QPS
 - Reservation Submission: 3 QPS

Step 2 - High-Level Design and API Design

RESTful APIs

Hotel APIs

- GET /v1/hotels/{id} - Get hotel details
- POST /v1/hotels - Add hotel (admin only)
- PUT /v1/hotels/{id} - Update hotel (admin only)
- DELETE /v1/hotels/{id} - Delete hotel (admin only)

Room APIs

- GET /v1/hotels/{id}/rooms/{roomId} - Get room details

- `POST /v1/hotels/{id}/rooms` - Add room (admin only)
- `PUT /v1/hotels/{id}/rooms/{roomId}` - Update room (admin only)
- `DELETE /v1/hotels/{id}/rooms/{roomId}` - Delete room (admin only)

Reservation APIs

- `GET /v1/reservations` - Get reservation history
- `GET /v1/reservations/{id}` - Get reservation details
- `POST /v1/reservations` - Create reservation
- `DELETE /v1/reservations/{id}` - Cancel reservation

High-Level Architecture

- **Client:** Web/Mobile App
- **API Gateway:** Routes requests to services, handles auth, rate limiting
- **Microservices:**
 - Hotel Service
 - Rate Service
 - Reservation Service
 - Payment Service
 - Hotel Management Service (admin-only)
- **Databases:** Each service has its own DB
- **CDN:** Caches static assets
- **Internal API:** Accessible only to admins (e.g., via VPN)

Data Model (Improved)

- **Entities:** hotel, room, room_type, guest, room_type_rate, reservation, room_type_inventory
- **Reservation tracks:** room_type_id, not room_id
- **Inventory Table:** tracks total_inventory and total_reserved per (hotel_id, room_type_id, date)
- **Reservation status:** pending, paid, refunded, canceled, rejected

Step 3 - Design Deep Dive

Concurrency Handling

Problem 1: Double Booking by Same User

Solution: Idempotent API using `reservation_id` as idempotency key (unique constraint avoids duplicate inserts)

Problem 2: Concurrent Bookings by Multiple Users

Solutions:

- **Pessimistic Locking:** `SELECT ... FOR UPDATE` (serializes access, not scalable)
- **Optimistic Locking:** use versioning to detect stale updates (retry needed under high contention)
- **Database Constraint:** CHECK constraint (simple, effective under low QPS)

Reservation Flow

1. User views availability
2. Client receives `reservation_id`
3. User submits booking with `reservation_id`
4. System reserves rooms using transaction-safe SQL

SQL Logic (Pseudocode)

```
-- Step 1: Check Inventory
SELECT total_inventory, total_reserved
FROM room_type_inventory
WHERE room_type_id = ? AND hotel_id = ? AND date BETWEEN ? AND ?;

-- Step 2: Reserve
UPDATE room_type_inventory
SET total_reserved = total_reserved + ?
WHERE room_type_id = ? AND hotel_id = ? AND date BETWEEN ? AND ?;
```

Support for Overbooking

```
if (total_reserved + numberToReserve <= 110% * total_inventory)
```

Scaling

- **Database Sharding:** by `hotel_id % N`
- **Caching Layer:** Redis with TTL, LRU eviction, async sync from DB
- **Change Data Capture (CDC):** Debezium to update cache from DB

Cache Inconsistency Mitigation

- All write logic must be validated against the DB (source of truth)

Microservice Data Consistency

- **Option 1 (used):** reservation & inventory in one DB for strong consistency
- **Alternative:** Saga pattern or 2PC for distributed transactions (adds complexity)

Step 4 - Wrap Up

Final Architecture Benefits

- **Scalable:** Stateless services, sharded databases
- **Resilient:** Caching, retries, database constraints
- **Consistent:** ACID transactions, idempotency, versioning

Visual Summary

- API Gateway connects clients to microservices
- Services communicate via gRPC or REST
- Each service owns its DB
- Inventory and reservations are validated and written transactionally
- Redis handles high-volume read traffic

Summary Table

Component	Responsibility
API Gateway	Auth, routing, rate limiting
Hotel Service	Static hotel and room info
Rate Service	Daily dynamic pricing
Reservation Svc	Reserve rooms, enforce constraints
Payment Service	Payment transaction + status update
Inventory DB	Total rooms, reserved rooms per day
Redis Cache	Fast access to inventory availability
Admin Interface	Internal hotel ops (refund, update)

